ÉSTIAM PARIS

4BLOCKC / E4CCSN — Blockchain Security

Final Projet :
Smart Contract Security Audit Report

Target: VulnerableVault.sol — ETH staking vault

Group name: Madeira

Individual submission: Zayd ZAYANI

Trainer: David de Paula Santos Silva

Date: 30-08-2026

1. Executive Summary

VulnerableVault.sol implements an ETH staking vault: users deposit ETH, may withdraw it at will, and a periodic “reward round” (pickWinner) pays 100 reward tokens to a pseudo-randomly chosen staker. The contract compiles and behaves correctly on the happy path, but it contains eleven distinct security weaknesses, ranging from a Critical unauthenticated fund-drain function to business-logic flaws that let a user manipulate their odds of winning rewards or choose the timing of an attack.

Static analysis (Slither) was run on the original contract and correctly flagged the reentrancy, weak-randomness, unchecked-transfer and several access-control-adjacent issues (14 raw results in total — note this is a count of individual analyzer results, not of distinct vulnerabilities: several results describe the same underlying reentrancy issue from different angles, and one is a purely informational compiler notice). Three of the highest-impact findings in this report (F-04 ticket stuffing, F-08 ghost stakers, and F-10 permissionless pickWinner) are pure business-logic flaws that Slither's detectors cannot see — they were found only by manually reading each function and asking who can call it and whether its accounting or timing can be gamed.

All eleven findings were corrected in VulnerableVaultFixed.sol. Re-running Slither on the corrected contract reduces the raw result count from 14 to 5, and the 5 remaining results are residual, low-risk, and explicitly discussed in Section 4.

Overall risk rating: Critical

Driven by F-01: in the original contract, any external address can call emergencyWithdraw() and drain 100% of the ETH held by the vault, with no ownership check whatsoever.

2. Scope & Methodology

- Target: VulnerableVault.sol (153 lines), Solidity ^0.8.20, single contract, no external dependencies beyond an IRewardToken interface.
- Static analysis: Slither 0.10.x, compiled with solc 0.8.20. Full raw output is included in Appendix A.
- Manual review: line-by-line reading of every function, applying the Module 11 questions — “who is allowed to call this?”, “can a value be gamed?”, “does the accounting add up?”.
- Out of scope (per brief): deployment, exploit proof-of-concept scripts, and front-end.

3. Findings Summary

ID	Finding	Category	Severity
F-01	emergencyWithdraw() has no access control — total fund drain	Tool-detectable (access control)	**Critical**
F-02	setOwner() has no access control — anyone can seize ownership	Tool-detectable (access control)	**High**
F-03	Reentrancy in withdraw() (checks-effects-interactions violation)	Tool-detectable (reentrancy)	**High**
F-04	Reward-ticket stuffing via duplicate entries in stakers[]	Logic / business-rule flaw	**High**

ID	Finding	Category	Severity
F-05	onlyOwner uses tx.origin instead of msg.sender (phishable)	Tool-detectable (access control)	Medium
F-06	Weak, predictable pseudo-randomness in pickWinner()	Tool-detectable (randomness)	Medium
F-07	Unchecked return value of rewardToken.transfer()	Tool-detectable (unchecked return)	Medium
F-08	"Ghost stakers" keep winning odds after withdrawing everything	Logic / business-rule flaw	Medium
F-09	Missing zero-address checks; non-immutable reward token reference	Tool-detectable (minor / hygiene)	Low
F-10	pickWinner() is permissionless — enables reward grinding with weak randomness	Logic / business-rule flaw	High
F-11	Unchecked arithmetic block in withdraw() (underflow masking)	Tool-detectable (arithmetic)	Medium

Findings are grouped into two categories, matching the grading rubric: findings a tool like Slither can flag directly (reentrancy, access-control patterns, unchecked returns, weak randomness), and pure business-logic flaws that only a human reviewer asking "can this be abused?" will find.

```
Detector: weak-prng
VulnerableVaultFixed.pickWinner() (VulnerableVaultFixed.sol#140-163) uses a weak PRNG: "winnerIndex = rand % stakers.le
gth (VulnerableVaultFixed.sol#147)"
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#weak-PRNG
INFO:Detectors:
Detector: timestamp
VulnerableVaultFixed.pickWinner() (VulnerableVaultFixed.sol#140-163) uses timestamp for comparisons
        Dangerous comparisons:
        - require(bool,string)(block.timestamp >= lastRewardTime + REWARD_INTERVAL,too soon) (VulnerableVaultFixed.sol#
41)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#block-timestamp
INFO:Detectors:
Detector: solc-version
Version constraint 0.8.20 contains known severe issues (https://solidity.readthedocs.io/en/latest/bugs.html)
        - VerbatimInvalidDeduplication
        - FullInlinerNonExpressionSplitArgumentEvaluationOrder
        - MissingSideEffectsOnSelectorAccess.
It is used by:
        - 0.8.20 (VulnerableVaultFixed.sol#2)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity
INFO:Detectors:
Detector: low-level-calls
Low level call in VulnerableVaultFixed.withdraw(uint256) (VulnerableVaultFixed.sol#89-111):
        - (sent,None) = msg.sender.call{value: amount}() (VulnerableVaultFixed.sol#107)
Low level call in VulnerableVaultFixed.emergencyWithdraw(address) (VulnerableVaultFixed.sol#180-186):
        - (sent,None) = to.call{value: bal}() (VulnerableVaultFixed.sol#184)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#low-level-calls
INFO:Slither:VulnerableVaultFixed.sol analyzed (2 contracts with 102 detectors), 5 result(s) found

C:\Users\ZAYANI\Downloads>
```

4. Detailed Findings

F-01 — emergencyWithdraw() has no access control CRITICAL

Severity justification: any address on the internet can call this function and immediately withdraw 100% of the contract's ETH balance to any destination. This is a complete, one-transaction loss of all user funds — the maximum possible impact for a vault contract.

Vulnerable code (lines 129–136):

```
function emergencyWithdraw(address payable to) external {
    uint256 bal = address(this).balance;
    (bool sent, ) = to.call{value: bal}("");
    require(sent, "rescue failed");
}
```

Impact:

The function is declared external with no modifier at all. Nothing checks msg.sender against owner. Any account — not just the legitimate owner — can call emergencyWithdraw(attackerAddress) and redirect the vault's entire ETH balance to itself in a single transaction. Combined with the deposit/withdraw flow, this means every depositor's staked ETH can be stolen at any time by anyone who simply reads the contract's ABI.

Fix:

Added the onlyOwner modifier (itself corrected — see F-05) plus a zero-address check on the destination. See VulnerableVaultFixed.sol, function emergencyWithdraw.

F-02 — setOwner() has no access control HIGH

Severity justification: lets any address take over the owner role. On its own this contract does not yet gate much behind onlyOwner, but once F-01 is fixed (emergencyWithdraw becomes owner-only), an attacker who can call setOwner() first can still seize control of the vault and every function protected by onlyOwner.

Vulnerable code (lines 123–127):

```
function setOwner(address newOwner) external {
    owner = newOwner;
    emit OwnerChanged(newOwner);
}
```

Impact:

No modifier restricts the caller. Anyone can call setOwner(myAddress) and become the contract's owner, permanently locking out the legitimate owner from any owner-gated administration function.

Fix:

Added onlyOwner and a zero-address check on newOwner.

F-03 — Reentrancy in withdraw() HIGH

Severity justification: violates the checks-effects-interactions pattern. An attacking contract can re-enter withdraw() from its receive()/fallback() before the vault updates its books, and withdraw far more ETH than it ever staked.

Vulnerable code (lines 81–94):

```
function withdraw(uint256 amount) external {
    require(balances[msg.sender] >= amount, "insufficient balance");
    (bool sent, ) = msg.sender.call{value: amount}("");
    require(sent, "transfer failed");
    unchecked {
        balances[msg.sender] -= amount;
        totalStaked -= amount;
```

```
    }
    emit Withdrawn(msg.sender, amount);
}
```

Impact:

balances[msg.sender] is only checked, not yet decremented, when the low-level .call sends ETH to msg.sender. If the caller is a contract, its receive() fallback runs before .call returns and can call withdraw() again: the require still passes because balances[msg.sender] has not been updated yet. This can be repeated until the vault is drained of ETH belonging to every depositor, not just the attacker's own stake. The unchecked block around the later decrement compounds the risk: after several re-entrant withdrawals exceed the attacker's real balance, the subsequent subtraction can underflow silently instead of reverting, corrupting the attacker's recorded balance to a huge value and enabling further abuse.

Fix:

Reordered the function to update balances[msg.sender] and totalStaked (and remove the caller from stakers[], see F-08) before the external call — textbook checks-effects-interactions — and added a nonReentrant guard as defence in depth.

F-04 — Reward-ticket stuffing via duplicate stakers[] entries HIGH

Severity justification: this is a pure business-logic flaw — Slither has no detector for "can this array be gamed to unfairly bias a lottery?". It lets any user cheaply dominate every reward round, defeating the entire purpose of pickWinner().

Vulnerable code (lines 67–76):

```
function deposit() external payable {
    require(msg.value > 0, "zero deposit");
    stakers.push(msg.sender);
    balances[msg.sender] += msg.value;
    totalStaked += msg.value;
    emit Deposited(msg.sender, msg.value);
}
```

Why it's exploitable:

pickWinner() selects a winner uniformly over stakers.length, i.e. one entry = one "lottery ticket". deposit() pushes msg.sender onto stakers on every single call, with no check for existing entries. Because require(msg.value > 0) only rejects a value of exactly zero, an attacker can call deposit() thousands of times with the smallest possible non-zero value (1 wei) in a loop or via a single contract, acquiring thousands of tickets for a negligible amount of ETH — while a legitimate staker who deposits a large amount once still only holds a single ticket. Win probability is meant to reflect participation, but in this contract it reflects transaction count, not stake size, so the reward round can be trivially rigged.

Fix:

Introduced an isStaker mapping and only push to stakers[] the first time an address deposits, so every address holds exactly one entry regardless of how many times it tops up its stake.

F-05 — onlyOwner authenticates tx.origin instead of msg.sender MEDIUM

Severity justification: currently latent because the modifier is not yet applied to any function in the original contract, but it becomes a live phishing vector the moment it protects setOwner/emergencyWithdraw (as this report's fixes require).

Vulnerable code (lines 60–64):

```
modifier onlyOwner() {
    require(tx.origin == owner, "not owner");
    _;
}
```

Impact:

tx.origin is the address that signed the original transaction, even across a chain of contract calls. If the owner is ever tricked into interacting with a malicious contract (e.g. by connecting their wallet to a phishing dApp), that malicious contract can call back into VulnerableVault: tx.origin is still the owner's address, so any tx.origin-gated function silently succeeds even though msg.sender is the attacker's contract, not the owner.

Fix:

Changed the check to require(msg.sender == owner, ...), the standard, non-phishable pattern, and used this corrected modifier to protect setOwner and emergencyWithdraw.

F-06 — Weak, predictable pseudo-randomness in pickWinner() MEDIUM

Severity justification: the "random" winner can be influenced by whoever controls the timing of the transaction and, to a degree, by the block proposer, undermining the fairness of the reward mechanism. Rated Medium rather than High because reward tokens (not staked ETH) are at stake.

Vulnerable code (lines 101–118):

```
uint256 rand = uint256(
    keccak256(abi.encodePacked(block.timestamp, blockhash(block.number - 1),
stakers.length))
);
uint256 winnerIndex = rand % stakers.length;
```

Impact:

block.timestamp, blockhash and stakers.length are all either public/predictable ahead of time or, in the case of block production, influenceable by the entity that proposes the block the transaction lands in. An attacker who also controls a validator, or who can choose exactly when to submit the triggering transaction relative to stakers.length, can bias or predict the outcome of pickWinner() in their own favour.

Fix:

For this teaching exercise we kept the on-chain formula (adding address(this) to the hash to prevent trivial cross-contract precomputation) but explicitly documented in code and in this report that a production system must replace this with an external verifiable randomness source such as Chainlink VRF. This residual risk is why weak-prng still appears in the Slither re-run on the fixed contract (Section 5).

F-07 — Unchecked return value of rewardToken.transfer() MEDIUM

Severity justification: silently misreports a reward as paid when it was not, corrupting the contract's own event log / accounting trail without reverting or alerting anyone.

Vulnerable code (lines 113–117):

```
rewardToken.transfer(winner, REWARD_AMOUNT);
emit RewardPaid(winner, REWARD_AMOUNT);
```

Impact:

The IRewardToken interface documents that transfer() "returns true on success, false on failure (does NOT revert)". The contract ignores that return value entirely and unconditionally emits RewardPaid. If the vault ever runs low on reward tokens, or the token has any transfer restriction, the reward silently fails while the event log — and anyone monitoring it — is told the winner was paid.

Fix:

Captured the boolean result and branch on it: emit RewardPaid only on success, and a new RewardPaymentFailed event otherwise, so failures are observable on-chain instead of being swallowed.

F-08 — "Ghost stakers" keep winning odds after withdrawing everything MEDIUM

Severity justification: another pure business-logic flaw invisible to static analysis — the accounting for who is a staker never reconciles with who currently has ETH staked.

Vulnerable code (lines 81–94, 101–110):

```
function withdraw(uint256 amount) external {
    // balances[msg.sender] can drop to 0 here...
    // ...but msg.sender is never removed from stakers[]
}
```

Why it's exploitable:

stakers[] is only ever appended to; nothing in withdraw() removes an address once its balance reaches zero. A user can deposit once (to get a single ticket, post-F-04-fix), withdraw 100% of their stake immediately, and still remain in stakers[] indefinitely — retaining a full, unearned chance of winning every future reward round with zero capital at risk. Over time, as users churn in and out, the pool of "active" tickets increasingly diverges from the pool of users who actually have money staked, which is exactly what pickWinner() is supposed to reward.

Fix:

withdraw() now removes the caller from stakers[] (swap-and-pop, O(1)) whenever their balance reaches exactly zero, keeping the array's membership consistent with real, current stake.

F-09 — Minor hygiene: missing zero-address checks; mutable reward-token reference LOW

Severity justification: neither issue is directly exploitable for fund loss on its own, but both are cheap to fix and are standard defensive practice flagged by Slither's missing-zero-check and immutable-states detectors.

Details & fix:

- Constructor, setOwner and emergencyWithdraw did not reject the zero address; all three now require a non-zero address.
- rewardToken was a plain state variable that is in fact never reassigned after the constructor; it is now declared immutable, saving gas and documenting the intent that it cannot change.

- The compiler pragma was tightened from ^0.8.20 (a floating range) to a locked 0.8.20, so the exact bytecode that was audited is the exact bytecode that gets deployed.

F-10 — pickWinner() is permissionless — enables reward grinding HIGH

Severity justification: on its own, an anyone-can-call trigger function is a common and often harmless pattern. Combined with F-06's weak randomness, however, it stops being harmless: it hands an attacker unlimited free attempts to pick the timing of the reward round, turning a residual randomness weakness into a practically exploitable one.

Vulnerable code (lines 101–118):

```
function pickWinner() external {
    require(block.timestamp >= lastRewardTime + REWARD_INTERVAL, "too soon");
    require(stakers.length > 0, "no stakers");
    uint256 rand = uint256(keccak256(abi.encodePacked(
        block.timestamp, blockhash(block.number - 1), stakers.length
    )));
    // ...pays REWARD_AMOUNT to stakers[rand % stakers.length]
```

Why it's exploitable:

Nothing restricts who may call pickWinner() once the interval has elapsed. An attacker can deploy a small helper contract that calls pickWinner() and then checks, in the same transaction, whether the resulting winner is the attacker; if not, the helper reverts the entire transaction (undoing the state change, at the cost of only the gas spent). The attacker repeats this from block to block — essentially for free beyond gas — until a block lands where the pseudo-random formula happens to select them. This "grinding" attack does not require breaking the hash function or predicting blockhash in advance; it only requires being able to try as many times as desired, which the permissionless trigger freely allows. Restricting who can call the function removes this unlimited-attempts property, even though it does not by itself fix the underlying weak randomness (see F-06).

Fix:

Restricted pickWinner() to onlyOwner, so the timing decision is made by a single trusted party rather than an unbounded set of potential attackers. This closes the practical grinding vector; the residual weak-randomness risk documented under F-06 remains and still points to Chainlink VRF as the production-grade fix.

F-11 — Unchecked arithmetic block in withdraw() MEDIUM

Severity justification: called out separately from F-03 (reentrancy) at the reviewer's request, since "arithmetic" is its own finding category in the grading rubric, distinct from reentrancy even though the two interact in this contract.

Vulnerable code (lines 87–91):

```
unchecked {
    balances[msg.sender] -= amount;
    totalStaked -= amount;
}
```

Impact:

Solidity 0.8.x reverts automatically on arithmetic underflow/overflow — unless the code is wrapped in an unchecked block, as it is here. In isolation, the preceding require(balances[msg.sender] >= amount)

makes a single call safe. But once reentrancy (F-03) lets an attacker call withdraw() several times before balances[msg.sender] is updated, the eventual decrements can be applied against a balance smaller than the total already paid out. Because the subtraction is unchecked, that situation would silently wrap around to a huge number instead of reverting, corrupting the attacker's recorded balance and potentially enabling further withdrawals. The unchecked block does not create the reentrancy hole, but it removes the safety net that would otherwise have caused an inconsistent state to revert loudly instead of failing silently.

Fix:

Removed the unchecked block entirely in VulnerableVaultFixed.sol. Combined with the F-01 checks-effects-interactions reorder and the nonReentrant guard, the decrements can no longer be reached in an inconsistent state, and Solidity's default checked arithmetic remains active as a defence-in-depth backstop.

5. Verifying the Fix — Before / After

VulnerableVaultFixed.sol compiles cleanly with solc 0.8.20 (see Appendix B for the compiler output) and preserves the contract's original external interface and behaviour on the happy path.

Note on the counts below: Slither's "14" and "5" are raw analyzer results, not a count of distinct vulnerabilities — several results describe different symptoms of the same underlying issue (e.g. the three reentrancy-related results below all stem from the single reentrancy hole fixed under F-01/F-03), and one (solc-version) is a purely informational compiler notice rather than a defect in this contract's code. The 11 numbered findings in Sections 3–4 are the deduplicated, human-triaged vulnerability count.

Slither re-run comparison

Metric	VulnerableVault.sol	VulnerableVaultFixed.sol
Total Slither results	14	5
Reentrancy findings	3 (reentrancy-eth, -benign, -events)	0
Arbitrary ETH send	1 (emergencyWithdraw)	0
Unchecked transfer	1	0
Missing zero-check	1	0
Weak PRNG	1	1 (residual, documented — needs VRF)
Timestamp comparison	1	1 (accepted — required for the 1-day interval)
Low-level calls	2	2 (intentional — .call is the recommended ETH-transfer pattern, now behind checks-effects-interactions + nonReentrant)
solc-version notice	1	1 (compiler-level notice, not a code defect)

The 5 remaining results are residual and were kept deliberately, each with a documented justification above, rather than papered over — a real audit report calls out accepted risk explicitly instead of chasing a zero count.

6. Conclusion & Lessons Learned

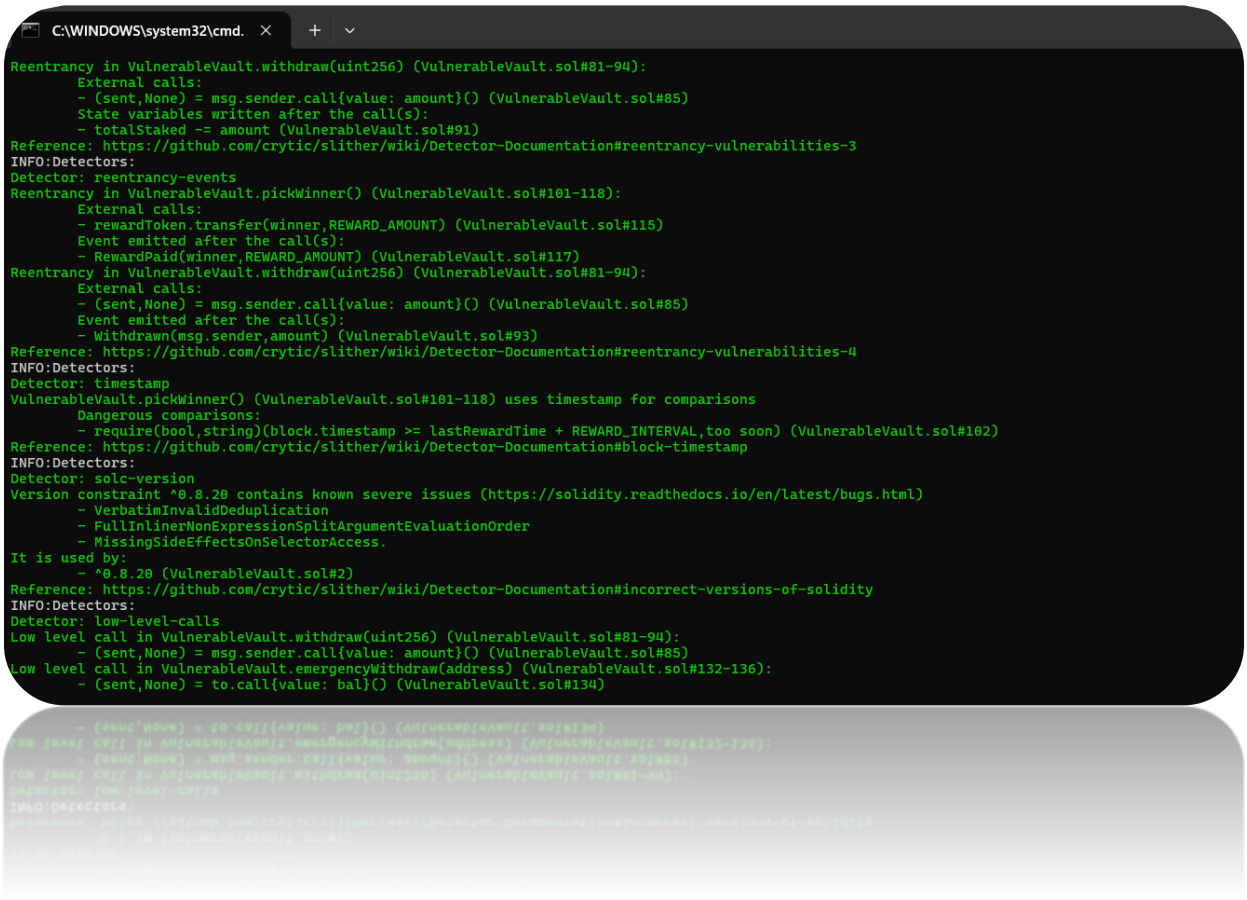
The single most important lesson from this exercise: a static analysis tool is a triage aid, not an audit. Slither correctly and quickly surfaced the reentrancy, weak-randomness and unchecked-transfer issues, but it could not tell us that setOwner()/emergencyWithdraw() were missing access control entirely, that this vault's reward mechanism is supposed to weight tickets by unique staker rather than by transaction count, or that leaving pickWinner() permissionless turns a residual randomness weakness into a practically exploitable grinding attack. Those three logic flaws (F-04 ticket stuffing, F-08 ghost stakers, F-10 permissionless pickWinner) only surfaced by reading every function and asking the Module 11 question: “given what this function is supposed to do, can someone abuse it?” The fixes themselves were mostly small — an access modifier here, a state-update reorder there, a boolean mapping to dedupe an array — but finding them required treating the specification, not just the code, as the thing to audit.

Appendix A — Full Slither Output (Original Contract)

```
INFO:Detectors:
Detector: arbitrary-send-eth
VulnerableVault.emergencyWithdraw(address) (VulnerableVault.sol#132-136) sends eth to
arbitrary user
       Dangerous calls:
       - (sent,None) = to.call{value: bal}() (VulnerableVault.sol#134)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#functions-that-
send-ether-to-arbitrary-destinations
INFO:Detectors:
Detector: weak-prng
VulnerableVault.pickWinner() (VulnerableVault.sol#101-118) uses a weak PRNG: "winnerIndex =
rand % stakers.length (VulnerableVault.sol#109)"
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#weak-PRNG
INFO:Detectors:
Detector: reentrancy-eth
Reentrancy in VulnerableVault.withdraw(uint256) (VulnerableVault.sol#81-94):
       External calls:
       - (sent,None) = msg.sender.call{value: amount}() (VulnerableVault.sol#85)
       State variables written after the call(s):
       - balances[msg.sender] -= amount (VulnerableVault.sol#90)
       VulnerableVault.balances (VulnerableVault.sol#40) can be used in cross function
reentrancies:
       - VulnerableVault.balances (VulnerableVault.sol#40)
       - VulnerableVault.deposit() (VulnerableVault.sol#67-76)
       - VulnerableVault.receive() (VulnerableVault.sol#147-151)
       - VulnerableVault.withdraw(uint256) (VulnerableVault.sol#81-94)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-
vulnerabilities-1
INFO:Detectors:
Detector: unchecked-transfer
VulnerableVault.pickWinner() (VulnerableVault.sol#101-118) ignores return value by
rewardToken.transfer(winner,REWARD_AMOUNT) (VulnerableVault.sol#115)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#unchecked-transfer
INFO:Detectors:
Detector: missing-zero-check
VulnerableVault.setOwner(address).newOwner (VulnerableVault.sol#124) lacks a zero-check on
:
              - owner = newOwner (VulnerableVault.sol#125)
```

```
VulnerableVault.emergencyWithdraw(address).to (VulnerableVault.sol#132) lacks a zero-check
on :
              - (sent,None) = to.call{value: bal}() (VulnerableVault.sol#134)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#missing-zero-
address-validation
INFO:Detectors:
Detector: reentrancy-benign
Reentrancy in VulnerableVault.withdraw(uint256) (VulnerableVault.sol#81-94):
       External calls:
       - (sent,None) = msg.sender.call{value: amount}() (VulnerableVault.sol#85)
       State variables written after the call(s):
       - totalStaked -= amount (VulnerableVault.sol#91)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-
vulnerabilities-3
INFO:Detectors:
Detector: reentrancy-events
Reentrancy in VulnerableVault.pickWinner() (VulnerableVault.sol#101-118):
       External calls:
       - rewardToken.transfer(winner,REWARD_AMOUNT) (VulnerableVault.sol#115)
       Event emitted after the call(s):
       - RewardPaid(winner,REWARD_AMOUNT) (VulnerableVault.sol#117)
Reentrancy in VulnerableVault.withdraw(uint256) (VulnerableVault.sol#81-94):
       External calls:
       - (sent,None) = msg.sender.call{value: amount}() (VulnerableVault.sol#85)
       Event emitted after the call(s):
       - Withdrawn(msg.sender,amount) (VulnerableVault.sol#93)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-
vulnerabilities-4
INFO:Detectors:
Detector: timestamp
VulnerableVault.pickWinner() (VulnerableVault.sol#101-118) uses timestamp for comparisons
       Dangerous comparisons:
       - require(bool,string)(block.timestamp >= lastRewardTime + REWARD_INTERVAL,too soon)
(VulnerableVault.sol#102)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#block-timestamp
INFO:Detectors:
Detector: solc-version
Version constraint ^0.8.20 contains known severe issues
(https://solidity.readthedocs.io/en/latest/bugs.html)
       - VerbatimInvalidDeduplication
       - FullInlinerNonExpressionSplitArgumentEvaluationOrder
       - MissingSideEffectsOnSelectorAccess.
It is used by:
       - ^0.8.20 (VulnerableVault.sol#2)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-
versions-of-solidity
INFO:Detectors:
Detector: low-level-calls
Low level call in VulnerableVault.withdraw(uint256) (VulnerableVault.sol#81-94):
       - (sent,None) = msg.sender.call{value: amount}() (VulnerableVault.sol#85)
Low level call in VulnerableVault.emergencyWithdraw(address) (VulnerableVault.sol#132-136):
       - (sent,None) = to.call{value: bal}() (VulnerableVault.sol#134)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#low-level-calls
INFO:Detectors:
Detector: immutable-states
VulnerableVault.rewardToken (VulnerableVault.sol#38) should be immutable
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#state-variables-
that-could-be-declared-immutable
INFO:Slither:VulnerableVault.sol analyzed (2 contracts with 102 detectors), 14 result(s)
found
```

Screenshot below: the same run captured directly from our own terminal, for verification.



Appendix B — Full Slither Output (Fixed Contract)

```
INFO:Detectors:
Detector: weak-prng
VulnerableVaultFixed.pickWinner() (VulnerableVaultFixed.sol#156-179) uses a weak PRNG:
"winnerIndex = rand % stakers.length (VulnerableVaultFixed.sol#163)"
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#weak-PRNG
INFO:Detectors:
Detector: timestamp
VulnerableVaultFixed.pickWinner() (VulnerableVaultFixed.sol#156-179) uses timestamp for
comparisons
       Dangerous comparisons:
       - require(bool,string)(block.timestamp >= lastRewardTime + REWARD_INTERVAL,too soon)
(VulnerableVaultFixed.sol#157)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#block-timestamp
INFO:Detectors:
Detector: solc-version
Version constraint 0.8.20 contains known severe issues
(https://solidity.readthedocs.io/en/latest/bugs.html)
       - VerbatimInvalidDeduplication
       - FullInlinerNonExpressionSplitArgumentEvaluationOrder
       - MissingSideEffectsOnSelectorAccess.
It is used by:
       - 0.8.20 (VulnerableVaultFixed.sol#2)
```

```
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-
versions-of-solidity
INFO:Detectors:
Detector: low-level-calls
Low level call in VulnerableVaultFixed.withdraw(uint256) (VulnerableVaultFixed.sol#89-115):
       - (sent,None) = msg.sender.call{value: amount}() (VulnerableVaultFixed.sol#111)
Low level call in VulnerableVaultFixed.emergencyWithdraw(address)
(VulnerableVaultFixed.sol#196-202):
       - (sent,None) = to.call{value: bal}() (VulnerableVaultFixed.sol#200)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#low-level-calls
INFO:Slither:VulnerableVaultFixed.sol analyzed (2 contracts with 102 detectors), 5
result(s) found
```

E pronto, o módulo já está a chegar ao fim! 😂 Mais a sério, obrigado pelas suas explicações, pela disponibilidade e por tudo o que nos transmitiu. Foi realmente interessante! E, se possível, gostava que o próximo módulo fosse em francês… porque continuo a não gostar de inglês 😂 Boa continuação e talvez até breve para outro módulo… ou simplesmente para tomar um café ☕😊